Bitácora de prompts - borrador para el equipo

Estado: borrador. El equipo debe contrastar los prompts reconstruidos con su historial real, agregar sus nombres y anexar capturas de la app y del repositorio. No presenten como conversación literal una reconstrucción.

Herramienta: OpenAI Codex. El nombre exacto del modelo no se muestra en este hilo.

Fecha de trabajo: 14 de octubre de 2026.

Registro de fases

Prompt 1 - Solicitud inicial

- Fase: 0-6, solicitud general del proyecto.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt enviado (literal): “ayudame a desarrollar esto”. Se adjuntó el enunciado `Parcial_2do_corte_Simulador_Automatas.pdf`.
- Respuesta (resumen): Se extrajeron los requisitos del parcial y se preparó un simulador local en Python/Tkinter con lógica separada, editor JSON, módulos AFND→AFD y autómata→ER, verificador, pruebas y documentación.
- Decisión del equipo: Pendiente de revisión y aprobación del equipo. La implementación tomó Python/Tkinter porque el enunciado lo recomienda.
- Verificación: Se cotejaron requisitos con el PDF y se implementaron los dos casos del Anexo A.

Fase 0 - Especificación

- N.º: 2.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de trabajo documentado: “Rol: analista de requisitos de software educativo. Contexto: parcial de Teoría de la Computación para un simulador local de autómatas. Tarea: redacta una especificación con entradas, salidas, conversión por subconjuntos, ecuaciones para ER, verificador y casos límite. Restricciones: Python/Tkinter o HTML local, sin servidor ni internet; contempla pozo, estados inalcanzables, alfabeto de un símbolo y ε opcional. Formato: SPEC.md con requisitos verificables. Verificación: compara cada requisito con el enunciado y el Anexo A.”
- Respuesta (resumen): Se creó `SPEC.md` con entradas/salidas, restricciones y criterios de aceptación.
- Decisión del equipo: Pendiente. Revisar la notación de λ/ε usada en clase.
- Verificación: Revisión manual frente a las secciones 2, 3 y Anexo A del enunciado.

Fase 1 - Arquitectura

- N.º: 3.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de trabajo documentado: “Rol: arquitecto de software. Contexto: el SPEC.md aprobado describe un simulador offline de autómatas. Tarea: propone estructura de archivos, modelo JSON y flujo de AFND a AFD a ER. Restricciones: un solo stack, biblioteca estándar, una app local. Formato: ARQUITECTURA.md con responsabilidades y decisiones pendientes. Verificación: el formato debe representar estados, alfabeto, inicial, finales, transiciones normales y ε.”
- Respuesta (resumen): Se propuso dividir interfaz, lógica, casos de referencia y pruebas; el AFND y el AFD comparten el mismo formato JSON.
- Decisión del equipo: Pendiente de aprobación. No se conoce todavía la composición del grupo.
- Verificación: El formato permite codificar los dos casos del anexo y transiciones ε.

Fase 2 - Lógica pura

- N.º: 4.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de trabajo documentado: “Rol: desarrollador senior de Python. Contexto: implementamos un simulador offline con el JSON del SPEC. Tarea: escribe funciones puras para validar el autómata, calcular clausuras-ε, convertir AFND a AFD por subconjuntos, formar y resolver ecuaciones de ER y verificar lenguajes. Restricciones: solo biblioteca estándar, estado pozo si se alcanza y funciones fáciles de explicar. Formato: automatas.py más pruebas unittest para ambos casos del Anexo A. Verificación: el caso 1 debe producir los subconjuntos del enunciado; el caso 2 debe ser equivalente hasta longitud 6.”
- Respuesta (resumen): Se implementó la lógica en `automatas.py` y se añadieron pruebas para los casos 1 y 2, pozo y ε.
- Decisión del equipo: Pendiente de revisión del código y de ejecución por cada integrante.
- Verificación: `python -m unittest discover -s tests -v`; resultado: 3 pruebas aprobadas.

Fase 3 - Interfaz gráfica

- N.º: 5.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de trabajo documentado: “Rol: desarrollador de interfaces Tkinter. Contexto: existe una biblioteca de lógica pura y el JSON del SPEC. Tarea: crea una app con editor de autómatas, diagramas con estados ovalados, inicial con flecha, finales con doble círculo, tabla de conversión y conexión al módulo de ER. Restricciones: Tkinter, sin red ni dependencias. Formato: app.py ejecutable con python app.py. Verificación: carga los casos del anexo y conserva legible el estado pozo.”
- Respuesta (resumen): Se creó `app.py` con pestañas, editor, grafos, tabla y botones para los casos de referencia.
- Decisión del equipo: Pendiente de comprobar visualmente en los computadores del grupo.
- Verificación: Compilación sintáctica y prueba de creación de la ventana con dos pestañas; falta una captura tomada por el equipo al ejecutar la app.

Fase 4 - Paso a paso

- N.º: 6.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de trabajo documentado: "Rol: tutor de Teoría de la Computación. Contexto: el simulador ya calcula transiciones y ecuaciones. Tarea: muestra cada subconjunto procesado, símbolo y destino; en ER, muestra el sistema inicial, regla aplicada, despeje y sustituciones. Restricciones: no ocultes los pasos detrás del resultado final. Formato: navegación anterior/siguiente y registro legible. Verificación: confirma que cada movimiento coincide con la tabla y que los despejes usan la regla de bucle/Arden o transición simple/múltiple."
- Respuesta (resumen): La primera pestaña navega los movimientos por símbolo; la segunda muestra sistema, reglas y sustituciones.
- Decisión del equipo: Pendiente de validar la terminología exacta enseñada en clase.
- Verificación: Casos 1 y 2 cargan desde botones para reproducir la demostración.

Fase 5 - Depuración

- N.º: 7.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de depuración 1 (reconstruido): "Analiza el fallo de la prueba del caso 2: una concatenación con una unión contiene referencias a estados en una alternativa no terminal. Identifica la causa, corrige la representación para mantener ecuaciones lineales y vuelve a ejecutar las pruebas."
- Síntoma: La resolución de ecuaciones falló con `No se pudo linealizar una ecuación`.
- Causa: El AST permitía `A*(B+C)`, pero el algoritmo de despeje esperaba alternativas distribuidas que terminaran en una sola variable.
- Corrección: Se distribuyó la concatenación sobre las alternativas de unión al construir el AST.
- Verificación: La prueba del caso 2 pudo continuar; las pruebas finales pasan.
- Prompt de depuración 2 (reconstruido): "Analiza el KeyError al sustituir una ecuación: el ciclo de sustitución intenta leer un estado que ya fue eliminado. Corrige el recorrido para usar únicamente ecuaciones restantes y ejecuta de nuevo las pruebas."
- Síntoma: `KeyError` para un estado ya eliminado al sustituir variables.
- Causa: La sustitución iteraba sobre todos los estados originales.
- Corrección: Se cambió el recorrido para iterar sobre las claves que aún permanecen en el sistema.
- Verificación: `python -m unittest discover -s tests -v`; resultado final: 3 pruebas aprobadas.
- Prompt de depuración 3 (reconstruido): "Analiza el error de arranque de Tkinter `expected integer but got UI`. Encuentra cómo se está interpretando la fuente global y ajusta la configuración sin cambiar los requisitos de interfaz. Verifica que la ventana y sus pestañas se creen."
- Síntoma: La inicialización de un control `Text` fallaba antes de mostrar la ventana.
- Causa: La opción global se pasó como la cadena `Segoe UI 10`; Tcl separó el nombre de fuente con espacios como parámetros distintos.
- Corrección: Se cambió la fuente global a `TkDefaultFont`; los widgets con fuentes explícitas conservan tuplas de familia y tamaño.
- Verificación: Prueba de humo: se crea la app, se actualizan los widgets y la ventana informa dos pestañas.

Fase 6 - Revisión y README

- N.º: 8.
- IA utilizada: OpenAI Codex; modelo no visible.
- Prompt de revisión documentado: "Rol: revisor de código Python. Contexto: app local Tkinter con algoritmos en automatas.py. Tarea: revisa que los módulos estén conectados, que el verificador no bloquee la interfaz y que las instrucciones de ejecución sean completas. Restricciones: no agregar dependencias ni servicios externos. Formato: lista de hallazgos y README. Verificación: ejecuta pruebas automáticas y compilación sintáctica."
- Respuesta (resumen): Se añadieron README con ejecución y alcance; el verificador usa un hilo de trabajo.
- Decisión del equipo: Pendiente de revisión final, prueba offline y commits por fase.
- Verificación: Tres pruebas aprobadas, `py_compile` sin errores y prueba de humo de la interfaz con dos pestañas.

Evidencia que debe completar el equipo antes de entregar

- Nombres de integrantes y quién revisó cada fase.
- Contrastar los prompts reconstruidos con el historial real de uso de IA y reemplazarlos por los textos literales enviados.
- Capturas de la aplicación ejecutándose, incluyendo los dos casos y el verificador.
- Commits con los cambios reales del equipo; no simular autores ni fechas.
- Video de máximo 3 minutos y evidencia de ejecución sin conexión.
- Confirmar individualmente que cada integrante puede explicar construcción de subconjuntos, Arden y la prueba de equivalencia.